

StatefulSet 基础

StatefulSet 基础

本教程介绍了如何使用

来管理应用。演示了如何创建、删除、扩容/缩容和更新 StatefulSet 的 Pod。

在开始本教程之前，你应该熟悉以下 Kubernetes 的概念：

- Pod
- Cluster DNS
- Headless Service
- PersistentVolume
- PersistentVolume 制备
- kubectl 命令行工具

你应该配置 kubectl 的上下文使用 default 命名空间。如果你使用的是现有集群，请确保可以使用该集群的 default 命名空间进行练习。理想情况下，在没有运行任何实际工作负载的集群中进行练习。

阅读有关 StatefulSet 的概念页面也很有用。

本教程假设你的集群被配置为动态制备 PersistentVolume 卷，且有一个默认 StorageClass。如果没有这样配置，在开始本教程之前，你需要手动准备 2 个 1 GiB 的存储卷，以便这些 PersistentVolume 可以映射到 StatefulSet 定义的 PersistentVolumeClaim 模板。

StatefulSet 旨在与有状态的应用及分布式系统一起使用。然而在 Kubernetes 上管理有状态应用和分布式系统是一个宽泛而复杂的话题。为了演示 StatefulSet 的基本特性，并且不使前后的主题混淆，你将会使用 StatefulSet 部署一个简单的 Web 应用。

在阅读本教程后，你将熟悉以下内容：

- 如何创建 StatefulSet

- StatefulSet 怎样管理它的 Pod
- 如何删除 StatefulSet
- 如何对 StatefulSet 进行扩容/缩容
- 如何更新一个 StatefulSet 的 Pod

创建 StatefulSet {#creating-a-statefulset}

作为开始，使用如下示例创建一个 StatefulSet（以及它所依赖的 Service）。它和 StatefulSet 概念中的示例相似。它创建了一个 Headless Service nginx 用来发布 StatefulSet web 中的 Pod 的 IP 地址。

你需要使用至少两个终端窗口。在第一个终端中，使用 `kubectl get` 来 StatefulSet 的 Pod 的创建情况。

```
# 使用此终端运行指定 --watch 的命令
# 当你被要求开始一个新的 watch 时结束这个 watch
kubectl get pods --watch -l app=nginx
```

在另一个终端中，使用 `kubectl apply` 来创建 Headless Service 和 StatefulSet。

```
kubectl apply -f https://k8s.io/examples/application/web/web.yaml
```

```
service/nginx created
statefulset.apps/web created
```

上面的命令创建了两个 Pod，每个都运行了一个 NGINX Web 服务器。获取 nginx Service：

```
kubectl get service nginx
```

```
NAME    TYPE    CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
nginx   ClusterIP  None        80/TCP      12s
```

然后获取 web StatefulSet，以验证两者均已成功创建：

```
kubectl get statefulset web
```

```
NAME    READY  AGE
web     2/2    37s
```

顺序创建 Pod {#ordered-pod-creation}

StatefulSet 默认以严格的顺序创建其 Pod。

对于一个拥有 **n** 个副本的 StatefulSet，Pod 被部署时是按照 **{0..n-1}** 的序号顺序创建的。在第一个终端中使用 `kubectl get` 检查输出。这个输出最终将看起来像下面的样子。

```
# 不要开始一个新的 watch
# 这应该已经处于 Running 状态
kubectl get pods --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	0/1	Pending	0	0s
web-0	0/1	Pending	0	0s
web-0	0/1	ContainerCreating	0	0s
web-0	1/1	Running	0	19s
web-1	0/1	Pending	0	0s
web-1	0/1	Pending	0	0s
web-1	0/1	ContainerCreating	0	0s
web-1	1/1	Running	0	18s

请注意，直到 web-0 Pod 处于 **Running**（请参阅 Pod 阶段）并 **Ready**（请参阅 Pod 状况中的 type）状态后，web-1 Pod 才会被启动。

在本教程的后面部分，你将练习并行启动。

要配置分配给 StatefulSet 中每个 Pod 的整数序号，请参阅起始序号。

StatefulSet 中的 Pod {#pods-in-a-statefulset}

StatefulSet 中的每个 Pod 拥有一个唯一的顺序索引和稳定的网络身份标识。

检查 Pod 的顺序索引 {#examining-the-pod-s-ordinal-index}

获取 StatefulSet 的 Pod：

```
kubectl get pods -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	1m
web-1	1/1	Running	0	1m

如同 StatefulSet 概念中所提到的，StatefulSet 中的每个 Pod 拥有一个具有黏性的、独一无二的身份标志。这个标志基于 StatefulSet 分配给每个 Pod 的唯一顺序索引。Pod 名称的格式为 -。web StatefulSet 拥有两个副本，所以它创建了两个 Pod：web-0 和 web-1。

使用稳定的网络身份标识 {#using-stable-network-identities}

每个 Pod 都拥有一个基于其顺序索引的稳定的主机名。使用 `kubectl exec` 在每个 Pod 中执行 `hostname`：

```
for i in 0 1; do kubectl exec "web-$i" -- sh -c 'hostname'; done  
  
web-0  
web-1
```

使用 `kubectl run` 运行一个提供 `nslookup` 命令的容器，该命令来自于 `dnsutils` 包。通过对 Pod 的主机名执行 `nslookup`，你可以检查这些主机名在集群内部的 DNS 地址：

```
kubectl run -i --tty --image busybox:1.28 dns-test --restart=Never --rm
```

这将启动一个新的 Shell。在新 Shell 中运行：

```
# 在 dns-test 容器 Shell 中运行以下命令  
nslookup web-0.nginx
```

输出类似于：

```
Server: 10.0.0.10  
Address 1: 10.0.0.10 kube-dns.kube-system.svc.cluster.local
```

```
Name: web-0.nginx  
Address 1: 10.244.1.6
```

```
nslookup web-1.nginx  
Server: 10.0.0.10  
Address 1: 10.0.0.10 kube-dns.kube-system.svc.cluster.local
```

```
Name: web-1.nginx  
Address 1: 10.244.2.6
```

（现在可以退出容器 Shell：exit）

Headless Service 的 CNAME 指向 SRV 记录（记录每个 Running 和 Ready 状态的 Pod）。SRV 记录指向一个包含 Pod IP 地址的记录表项。

在一个终端中监视 StatefulSet 的 Pod：

```
# 启动一个新的 watch
# 当你看到删除完成后结束这个 watch
kubectl get pod --watch -l app=nginx
```

在另一个终端中使用 `kubectl delete` 删除 StatefulSet 中所有的 Pod：

```
kubectl delete pod -l app=nginx
```

```
pod "web-0" deleted
```

```
pod "web-1" deleted
```

等待 StatefulSet 重启它们，并且两个 Pod 都变成 Running 和 Ready 状态：

```
# 这应该已经处于 Running 状态
kubectl get pod --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	0/1	ContainerCreating	0	0s

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	2s
web-1	0/1	Pending	0	0s
web-1	0/1	Pending	0	0s
web-1	0/1	ContainerCreating	0	0s
web-1	1/1	Running	0	34s

使用 `kubectl exec` 和 `kubectl run` 查看 Pod 的主机名和集群内部的 DNS 表项。首先，查看 Pod 的主机名：

```
for i in 0 1; do kubectl exec web-$i -- sh -c 'hostname'; done
```

```
web-0
```

```
web-1
```

然后，运行：

```
kubectl run -i --tty --image busybox:1.28 dns-test --restart=Never --rm
```

这将启动一个新的 Shell。在新 Shell 中，运行：

```
# 在 dns-test 容器 Shell 中运行以下命令
nslookup web-0.nginx
```

输出类似于：

```
Server: 10.0.0.10
Address 1: 10.0.0.10 kube-dns.kube-system.svc.cluster.local
```

```
Name: web-0.nginx
Address 1: 10.244.1.7
```

```
nslookup web-1.nginx
Server: 10.0.0.10
Address 1: 10.0.0.10 kube-dns.kube-system.svc.cluster.local
```

```
Name: web-1.nginx
Address 1: 10.244.2.8
```

（现在可以退出容器 Shell：exit）

Pod 的序号、主机名、SRV 条目和记录名称没有改变，但和 Pod 相关联的 IP 地址可能发生了改变。在本教程中使用的集群中它们就改变了。这就是为什么不要在其他应用中使用 StatefulSet 中特定 Pod 的 IP 地址进行连接，这点很重要（可以通过解析 Pod 的主机名来连接到 Pod）。

发现 StatefulSet 中特定的 Pod {#discovery-for-specific-pods-in-a-statefulset}

如果你需要查找并连接一个 StatefulSet 的活动成员，你应该查询 Headless Service 的 CNAME（nginx.default.svc.cluster.local）。和 CNAME 相关联的 SRV 记录只会包含 StatefulSet 中处于 Running 和 Ready 状态的 Pod。

如果你的应用已经实现了用于测试是否已存活（liveness）并就绪（readiness）的连接逻辑，你可以使用 Pod 的 SRV 记录（web-0.nginx.default.svc.cluster.local、web-1.nginx.default.svc.cluster.local）。因为它们是稳定的，并且当你的 Pod 的状态变为 Running 和 Ready 时，你的应用就能够发现它们的地址。

如果你的应用程序想要在 StatefulSet 中找到任一健康的 Pod，且不需要跟踪每个特定的 Pod，你还可以连接到由该 StatefulSet 中的 Pod 关联的 type: ClusterIP Service 的 IP 地址。你可以使用跟踪 StatefulSet 的同一 Service（StatefulSet 中 serviceName 所指定的）或选择正确的 Pod 集的单独 Service。

写入稳定的存储 {#writing-to-stable-storage}

获取 web-0 和 web-1 的 PersistentVolumeClaim：

```
kubectl get pvc -l app=nginx
```

输出类似于：

NAME	STATUS	VOLUME	CAPACITY	ACCESSMODES	AGE
www-web-0	Bound	pvc-15c268c7-b507-11e6-932f-42010a800002	1Gi	RWO	48s
www-web-1	Bound	pvc-15c79307-b507-11e6-932f-42010a800002	1Gi	RWO	48s

StatefulSet 控制器创建了两个，绑定到两个。

由于本教程使用的集群配置为动态制备 PersistentVolume 卷，所有的 PersistentVolume 卷都是自动创建和绑定的。

NGINX Web 服务器默认会加载位于 /usr/share/nginx/html/index.html 的 index 文件。StatefulSet spec 中的 volumeMounts 字段保证了 /usr/share/nginx/html 文件夹由一个 PersistentVolume 卷支持。

将 Pod 的主机名写入它们的 index.html 文件并验证 NGINX Web 服务器使用该主机名提供服务：

```
for i in 0 1; do kubectl exec "web-$(i)" -- sh -c 'echo "$(hostname)" > /usr/share/nginx/html/index.html'; done
```

```
for i in 0 1; do kubectl exec -i -t "web-$(i)" -- curl http://localhost/; done
```

```
web-0
web-1
```

请注意，如果你看见上面的 curl 命令返回了 **403 Forbidden** 的响应，你需要像这样修复使用 volumeMounts （原因归咎于使用 hostPath 卷时存在的缺陷）挂载的目录的权限，先运行：

```
for i in 0 1; do kubectl exec web-$(i) -- chmod 755 /usr/share/nginx/html; done
```

再重新尝试上面的 curl 命令。

在一个终端监视 StatefulSet 的 Pod：

```
kubectl get pod -w -l app=nginx
```

在另一个终端删除 StatefulSet 所有的 Pod：

```
# 当你到达该部分的末尾时结束此 watch
# 在开始“扩展 StatefulSet”时，你将启动一个新的 watch。
kubectl get pod --watch -l app=nginx
```

```
pod "web-0" deleted
pod "web-1" deleted
```

在第一个终端里检查 `kubectl get` 命令的输出，等待所有 Pod 变成 Running 和 Ready 状态。

```
# 这应该已经处于 Running 状态
kubectl get pod --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	0/1	ContainerCreating	0	0s

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	2s
web-1	0/1	Pending	0	0s
web-1	0/1	Pending	0	0s
web-1	0/1	ContainerCreating	0	0s
web-1	1/1	Running	0	34s

验证所有 Web 服务器在继续使用它们的主机名提供服务：

```
for i in 0 1; do kubectl exec -i -t "web-$(i)" -- curl http://localhost/; done
```

```
web-0
web-1
```

虽然 web-0 和 web-1 被重新调度了，但它们仍然继续监听各自的主机名，因为和它们的 PersistentVolumeClaim 相关联的 PersistentVolume 卷被重新挂载到了各自的 volumeMount 上。不管 web-0 和 web-1 被调度到了哪个节点上，它们的 PersistentVolume 卷将会被挂载到合适的挂载点上。

扩容/缩容 StatefulSet {#scaling-a-statefulset}

扩容/缩容 StatefulSet 指增加或减少它的副本数。这通过更新 replicas 字段完成（水平缩放）。你可以使用 `kubectl scale` 或者 `kubectl patch` 来扩容/缩容一个 StatefulSet。

扩容 {#scaling-up}

扩容意味着添加更多副本。如果你的应用程序能够在整个 StatefulSet 范围内分派工作，则新的更大的 Pod 集可以执行更多的工作。

在一个终端窗口监视 StatefulSet 的 Pod：

```
# 如果你已经有一个正在运行的 watch，你可以继续使用它。
# 否则，就启动一个。
```

```
# 当 StatefulSet 有 5 个健康的 Pod 时结束此 watch
kubectl get pods --watch -l app=nginx
```

在另一个终端窗口使用 `kubectl scale` 扩展副本数为 5:

```
kubectl scale sts web --replicas=5
```

```
statefulset.apps/web scaled
```

在第一个终端中检查 `kubectl get` 命令的输出，等待增加的 3 个 Pod 的状态变为 Running 和 Ready。

```
# 这应该已经处于 Running 状态
kubectl get pod --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	2h
web-1	1/1	Running	0	2h

NAME	READY	STATUS	RESTARTS	AGE
web-2	0/1	Pending	0	0s
web-2	0/1	Pending	0	0s
web-2	0/1	ContainerCreating	0	0s
web-2	1/1	Running	0	19s
web-3	0/1	Pending	0	0s
web-3	0/1	Pending	0	0s
web-3	0/1	ContainerCreating	0	0s
web-3	1/1	Running	0	18s
web-4	0/1	Pending	0	0s
web-4	0/1	Pending	0	0s
web-4	0/1	ContainerCreating	0	0s
web-4	1/1	Running	0	19s

StatefulSet 控制器扩展了副本的数量。如同创建 StatefulSet 所述，StatefulSet 按序号索引顺序创建各个 Pod，并且会等待前一个 Pod 变为 Running 和 Ready 才会启动下一个 Pod。

缩容 {#scaling-down}

缩容意味着减少副本数量。例如，你可能因为服务的流量水平已降低并且在当前规模下存在空闲资源的原因执行缩容操作。

在一个终端监视 StatefulSet 的 Pod:

```
kubectl get pods -w -l app=nginx
# 当 StatefulSet 只有 3 个 Pod 时结束此 watch
kubectl get pod --watch -l app=nginx
```

在另一个终端使用 kubectl patch 将 StatefulSet 缩容回三个副本：

```
kubectl patch sts web -p '{"spec":{"replicas":3}}'
```

```
statefulset.apps/web patched
```

等待 web-4 和 web-3 状态变为 Terminating。

```
kubectl get pods -w -l app=nginx
# 这应该已经处于 Running 状态
kubectl get pods --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	3h
web-1	1/1	Running	0	3h
web-2	1/1	Running	0	55s
web-3	1/1	Running	0	36s
web-4	0/1	ContainerCreating	0	18s

NAME	READY	STATUS	RESTARTS	AGE
web-4	1/1	Running	0	19s
web-4	1/1	Terminating	0	24s
web-4	1/1	Terminating	0	24s
web-3	1/1	Terminating	0	42s
web-3	1/1	Terminating	0	42s

顺序终止 Pod {#ordered-pod-termination}

控制器会按照与 Pod 序号索引相反的顺序每次删除一个 Pod。在删除下一个 Pod 前会等待上一个被完全关闭。

获取 StatefulSet 的 PersistentVolumeClaim：

```
kubectl get pvc -l app=nginx
```

NAME	STATUS	VOLUME	CAPACITY	ACCESSMODES	AGE
www-web-0	Bound	pvc-15c268c7-b507-11e6-932f-42010a800002	1Gi	RWO	13h
www-web-1	Bound	pvc-15c79307-b507-11e6-932f-42010a800002	1Gi	RWO	13h
www-web-2	Bound	pvc-e1125b27-b508-11e6-932f-42010a800002	1Gi	RWO	13h
www-web-3	Bound	pvc-e1176df6-b508-11e6-932f-42010a800002	1Gi	RWO	13h
www-web-4	Bound	pvc-e11bb5f8-b508-11e6-932f-42010a800002	1Gi	RWO	13h

五个 PersistentVolumeClaim 和五个 PersistentVolume 卷仍然存在。查看 Pod 的稳定存储，你会发现当删除 StatefulSet 的 Pod 时，挂载到 StatefulSet 的 Pod 的 PersistentVolume 卷不会被删除。当这种删除行为是由 StatefulSet 缩容引起时也是一样的。

更新 StatefulSet {#updating-statefulsets}

StatefulSet 控制器支持自动更新。更新策略由 StatefulSet API 对象的 spec.updateStrategy 字段决定。这个特性能够用来更新一个 StatefulSet 中 Pod 的容器镜像、资源请求和限制、标签和注解。

有两个有效的更新策略：RollingUpdate（默认）和 OnDelete。

滚动更新 {#rolling-update}

RollingUpdate 更新策略会更新一个 StatefulSet 中的所有 Pod，采用与序号索引相反的顺序并遵循 StatefulSet 的保证。

你可以通过指定 .spec.updateStrategy.rollingUpdate.partition 将使用 RollingUpdate 策略的 StatefulSet 的更新拆分为多个**分区**。你将在本教程中稍后练习此操作。

首先，尝试一个简单的滚动更新。

在一个终端窗口中对 web StatefulSet 执行 patch 操作来再次改变容器镜像：

```
kubectl patch statefulset web --type='json' -p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/image", "value": "registry.k8s.io/nginx-slim:0.24"}]'
statefulset.apps/web patched
```

在另一个终端监控 StatefulSet 中的 Pod：

```
# 滚动完成后结束此 watch
#
# 如果你不确定，请让它再运行一分钟
kubectl get pod -l app=nginx --watch
```

输出类似于：

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	7m
web-1	1/1	Running	0	7m
web-2	1/1	Running	0	8m
web-2	1/1	Terminating	0	8m

```

web-2  1/1  Terminating 0    8m
web-2  0/1  Terminating 0    8m
web-2  0/1  Terminating 0    8m
web-2  0/1  Terminating 0    8m
web-2  0/1  Terminating 0    8m
web-2  0/1  Pending 0    0s
web-2  0/1  Pending 0    0s
web-2  0/1  ContainerCreating 0    0s
web-2  1/1  Running 0    19s
web-1  1/1  Terminating 0    8m
web-1  0/1  Terminating 0    8m
web-1  0/1  Terminating 0    8m
web-1  0/1  Terminating 0    8m
web-1  0/1  Pending 0    0s
web-1  0/1  Pending 0    0s
web-1  0/1  ContainerCreating 0    0s
web-1  1/1  Running 0    6s
web-0  1/1  Terminating 0    7m
web-0  1/1  Terminating 0    7m
web-0  0/1  Terminating 0    7m
web-0  0/1  Terminating 0    7m
web-0  0/1  Terminating 0    7m
web-0  0/1  Terminating 0    7m
web-0  0/1  Pending 0    0s
web-0  0/1  Pending 0    0s
web-0  0/1  ContainerCreating 0    0s
web-0  1/1  Running 0    10s

```

StatefulSet 里的 Pod 采用和序号相反的顺序更新。在更新下一个 Pod 前，StatefulSet 控制器终止每个 Pod 并等待它们变成 Running 和 Ready。请注意，虽然在顺序后继者变成 Running 和 Ready 之前 StatefulSet 控制器不会更新下一个 Pod，但它仍然会重建任何在更新过程中发生故障的 Pod，使用的是它们现有的版本。

已经接收到更新请求的 Pod 将会被恢复为更新的版本，没有收到请求的 Pod 则会被恢复为之前的版本。像这样，控制器尝试继续使应用保持健康并在出现间歇性故障时保持更新的一致性。

获取 Pod 来查看它们的容器镜像：

```

for p in 0 1 2; do kubectl get pod "web-$p" --template '{{range $i, $c := .spec.containers}}
{{$.image}}{{end}}'; echo; done

```

```
registry.k8s.io/nginx-slim:0.24
registry.k8s.io/nginx-slim:0.24
registry.k8s.io/nginx-slim:0.24
```

StatefulSet 中的所有 Pod 现在都在运行之前的容器镜像。

你还可以使用 `kubectl rollout status sts/` 来查看 StatefulSet 的滚动更新状态。

分段更新 {#staging-an-update}

你可以通过指定 `.spec.updateStrategy.rollingUpdate.partition` 将使用 RollingUpdate 策略的 StatefulSet 的更新拆分为多个**分区**。

有关更多上下文，你可以阅读 StatefulSet 概念页面中的分区滚动更新。

你可以使用 `.spec.updateStrategy.rollingUpdate` 中的 `partition` 字段对 StatefulSet 执行更新的分段操作。对于此更新，你将保持 StatefulSet 中现有 Pod 不变，同时更改 StatefulSet 的 Pod 模板。然后，你（或通过教程之外的一些外部自动化工具）可以触发准备好的更新。

对 web StatefulSet 执行 patch 操作，为 `updateStrategy` 字段添加一个分区：

```
# "partition" 的值决定更改适用于哪些序号
# 确保使用比 StatefulSet 的最后一个序号更大的数字
kubectl patch statefulset web -p '{"spec":{"updateStrategy":{"type":"OnDelete",
"rollingUpdate": null}}}'
```

```
statefulset.apps/web patched
```

再次 patch StatefulSet 来改变此 StatefulSet 使用的容器镜像：

```
kubectl patch statefulset web --type='json' -p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/image", "value": "registry.k8s.io/nginx-slim:0.21"}]'
```

```
statefulset.apps/web patched
```

删除 StatefulSet 中的 Pod：

```
kubectl delete pod web-2
```

```
pod "web-2" deleted
```

等待替代的 Pod 变成 Running 和 Ready。

```
# 当你看到 web-2 运行正常时结束 watch
kubectl get pod -l app=nginx --watch
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	4m
web-1	1/1	Running	0	4m
web-2	0/1	ContainerCreating	0	11s
web-2	1/1	Running	0	18s

获取 Pod 的容器镜像：

```
kubectl get pod web-2 --template '{{range $i, $c := .spec.containers}}{{ $c.image}}{{end}}'
registry.k8s.io/nginx-slim:0.24
```

请注意，虽然更新策略是 RollingUpdate，StatefulSet 还是会使用原始的容器镜像恢复 Pod。这是因为 Pod 的序号比 updateStrategy 指定的 partition 更小。

金丝雀发布 {#rolling-out-a-canary}

现在，你将尝试对分段的变更进行金丝雀发布。

你可以通过减少上文指定的 partition 来进行金丝雀发布，以测试修改后的模板。

通过 patch 命令修改 StatefulSet 来减少分区：

```
# “partition” 的值应与 StatefulSet 现有的最高序号相匹配
kubectl patch statefulset web -p '{"spec":{"updateStrategy":
{"type":"RollingUpdate","rollingUpdate":{"partition":2}}}'
statefulset.apps/web patched
```

控制平面会触发 web-2 的替换（先优雅地删除现有 Pod，然后在删除完成后创建一个新的 Pod）。等待新的 web-2 Pod 变成 Running 和 Ready。

```
# 这应该已经处于 Running 状态
kubectl get pod -l app=nginx --watch
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	4m
web-1	1/1	Running	0	4m
web-2	0/1	ContainerCreating	0	11s
web-2	1/1	Running	0	18s

获取 Pod 的容器：

```
kubectl get pod web-2 --template '{{range $i, $c := .spec.containers}}{{ $c.image}}{{end}}'
registry.k8s.io/nginx-slim:0.21
```

当你改变 partition 时，StatefulSet 会自动更新 web-2 Pod，这是因为 Pod 的序号大于或等于 partition。

删除 web-1 Pod：

```
kubectl delete pod web-1
```

```
pod "web-1" deleted
```

等待 web-1 变成 Running 和 Ready。

这应该已经处于 Running 状态

```
kubectl get pod -l app=nginx --watch
```

输出类似于：

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	6m
web-1	0/1	Terminating	0	6m
web-2	1/1	Running	0	2m
web-1	0/1	Terminating	0	6m
web-1	0/1	Terminating	0	6m
web-1	0/1	Terminating	0	6m
web-1	0/1	Pending	0	0s
web-1	0/1	Pending	0	0s
web-1	0/1	ContainerCreating	0	0s
web-1	1/1	Running	0	18s

获取 web-1 Pod 的容器镜像：

```
kubectl get pod web-1 --template '{{range $i, $c := .spec.containers}}{{ $c.image}}{{end}}'
```

```
registry.k8s.io/nginx-slim:0.24
```

web-1 被按照原来的配置恢复，因为 Pod 的序号小于分区。当指定了分区时，如果更新了 StatefulSet 的 .spec.template，则所有序号大于或等于分区的 Pod 都将被更新。如果一个序号小于分区的 Pod 被删除或者终止，它将被按照原来的配置恢复。

分阶段的发布 {#phased-roll-outs}

你可以使用类似金丝雀发布的方法执行一次分阶段的发布（例如一次线性的、等比的或者指数形式的发布）。要执行一次分阶段的发布，你需要设置 partition 为希望控制器暂停更新的序号。

分区当前为 2，请将其设置为 0：

```
kubectl patch statefulset web -p '{"spec":{"updateStrategy":
{"type":"RollingUpdate","rollingUpdate":{"partition":0}}}'
```

statefulset.apps/web patched

等待 StatefulSet 中的所有 Pod 变成 Running 和 Ready。

这应该已经处于 Running 状态

```
kubectl get pod -l app=nginx --watch
```

输出类似于：

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	3m
web-1	0/1	ContainerCreating	0	11s
web-2	1/1	Running	0	2m
web-1	1/1	Running	0	18s
web-0	1/1	Terminating	0	3m
web-0	1/1	Terminating	0	3m
web-0	0/1	Terminating	0	3m
web-0	0/1	Terminating	0	3m
web-0	0/1	Terminating	0	3m
web-0	0/1	Terminating	0	3m
web-0	0/1	Pending	0	0s
web-0	0/1	Pending	0	0s
web-0	0/1	ContainerCreating	0	0s
web-0	1/1	Running	0	3s

获取 StatefulSet 中 Pod 的容器镜像详细信息：

```
for p in 0 1 2; do kubectl get pod "web-$p" --template '{{range $i, $c := .spec.containers}}
{{$c.image}}{{end}}'; echo; done
```

registry.k8s.io/nginx-slim:0.21

registry.k8s.io/nginx-slim:0.21

registry.k8s.io/nginx-slim:0.21

将 partition 改变为 0 以允许 StatefulSet 继续更新过程。

OnDelete 策略 {#on-delete}

通过将 .spec.updateStrategy.type 设置为 OnDelete，你可以为 StatefulSet 选择此更新策略。

对 web StatefulSet 执行 patch 操作，以使用 OnDelete 更新策略：

```
kubectl patch statefulset web -p '{"spec":{"updateStrategy":{"type":"OnDelete",  
"rollingUpdate": null}}}'
```

```
statefulset.apps/web patched
```

当你选择这个更新策略并修改 StatefulSet 的 .spec.template 字段时，StatefulSet 控制器将不会自动更新 Pod。你需要自己手动管理发布，或使用单独的自动化工具来管理发布。

删除 StatefulSet {#deleting-statefulsets}

StatefulSet 同时支持**非级联**和**级联**删除。使用非级联方式删除 StatefulSet 时，StatefulSet 的 Pod 不会被删除。使用级联删除时，StatefulSet 和它的 Pod 都会被删除。

阅读在集群中使用级联删除，以了解通用的级联删除。

非级联删除 {#non-cascading-delete}

在一个终端窗口监视 StatefulSet 中的 Pod。

```
# 当 StatefulSet 没有 Pod 时结束此 watch  
kubectl get pods --watch -l app=nginx
```

使用 kubectl delete 删除 StatefulSet。请确保提供了 --cascade=orphan 参数给命令。这个参数告诉 Kubernetes 只删除 StatefulSet 而**不要**删除它的任何 Pod。

```
kubectl delete statefulset web --cascade=orphan
```

```
statefulset.apps "web" deleted
```

获取 Pod 来检查它们的状态：

```
kubectl get pods -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	6m
web-1	1/1	Running	0	7m
web-2	1/1	Running	0	5m

虽然 web 已经被删除了，但所有 Pod 仍然处于 Running 和 Ready 状态。删除 web-0：

```
kubectl delete pod web-0
```

pod "web-0" deleted

获取 StatefulSet 的 Pod:

```
kubectl get pods -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-1	1/1	Running	0	10m
web-2	1/1	Running	0	7m

由于 web StatefulSet 已经被删除, web-0 没有被重新启动。

在一个终端监控 StatefulSet 的 Pod。

让 watch 一直运行到你下次启动 watch 为止

```
kubectl get pods --watch -l app=nginx
```

在另一个终端里重新创建 StatefulSet。请注意, 除非你删除了 nginx Service (你不应该这样做), 你将会看到一个错误, 提示 Service 已经存在。

```
kubectl apply -f https://k8s.io/examples/application/web/web.yaml
```

```
statefulset.apps/web created
```

```
service/nginx unchanged
```

请忽略这个错误。它仅表示 kubernetes 进行了一次创建 **nginx** Headless Service 的尝试, 尽管那个 Service 已经存在。

在第一个终端中运行并检查 kubectl get 命令的输出。

这应该已经处于 Running 状态

```
kubectl get pods --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-1	1/1	Running	0	16m
web-2	1/1	Running	0	2m

NAME	READY	STATUS	RESTARTS	AGE
web-0	0/1	Pending	0	0s
web-0	0/1	Pending	0	0s
web-0	0/1	ContainerCreating	0	0s
web-0	1/1	Running	0	18s
web-2	1/1	Terminating	0	3m
web-2	0/1	Terminating	0	3m
web-2	0/1	Terminating	0	3m
web-2	0/1	Terminating	0	3m

当重新创建 web StatefulSet 时，web-0 被第一个重新启动。由于 web-1 已经处于 Running 和 Ready 状态，当 web-0 变成 Running 和 Ready 时，StatefulSet 会接收这个 Pod。由于你重新创建的 StatefulSet 的 replicas 等于 2，一旦 web-0 被重新创建并且 web-1 被认为已经处于 Running 和 Ready 状态时，web-2 将会被终止。

现在再看看被 Pod 的 Web 服务器加载的 index.html 的内容：

```
for i in 0 1; do kubectl exec -i -t "web-$(i)" -- curl http://localhost/; done
```

web-0

web-1

尽管你同时删除了 StatefulSet 和 web-0 Pod，但它仍然使用最初写入 index.html 文件的主机名进行服务。这是因为 StatefulSet 永远不会删除和一个 Pod 相关联的 PersistentVolume 卷。当你重建这个 StatefulSet 并且重新启动了 web-0 时，它原本的 PersistentVolume 卷会被重新挂载。

级联删除 {#cascading-delete}

在一个终端窗口监视 StatefulSet 里的 Pod。

让它运行直到下一页部分

```
kubectl get pods --watch -l app=nginx
```

在另一个窗口中再次删除这个 StatefulSet，这次省略 --cascade=orphan 参数。

```
kubectl delete statefulset web
```

```
statefulset.apps "web" deleted
```

在第一个终端检查 kubectl get 命令的输出，并等待所有的 Pod 变成 Terminating 状态。

这应该已经处于 Running 状态

```
kubectl get pods --watch -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Running	0	11m
web-1	1/1	Running	0	27m

NAME	READY	STATUS	RESTARTS	AGE
web-0	1/1	Terminating	0	12m
web-1	1/1	Terminating	0	29m
web-0	0/1	Terminating	0	12m
web-0	0/1	Terminating	0	12m
web-0	0/1	Terminating	0	12m

```
web-1 0/1 Terminating 0 29m
web-1 0/1 Terminating 0 29m
web-1 0/1 Terminating 0 29m
```

如同你在扩容章节看到的，这些 Pod 按照与其序号索引相反的顺序每次终止一个。在终止一个 Pod 前，StatefulSet 控制器会等待 Pod 后继者被完全终止。

尽管级联删除会删除 StatefulSet 及其 Pod，但级联**不会**删除与 StatefulSet 关联的 Headless Service。你必须手动删除 nginx Service。

```
kubectl delete service nginx
```

```
service "nginx" deleted
```

再一次重新创建 StatefulSet 和 Headless Service：

```
kubectl apply -f https://k8s.io/examples/application/web/web.yaml
```

```
service/nginx created
statefulset.apps/web created
```

当 StatefulSet 所有的 Pod 变成 Running 和 Ready 时，获取它们的 index.html 文件的内容：

```
for i in 0 1; do kubectl exec -i -t "web-$(i)" -- curl http://localhost/; done
```

```
web-0
web-1
```

即使你已经删除了 StatefulSet 和它的全部 Pod，这些 Pod 将会被重新创建并挂载它们的 PersistentVolume 卷，并且 web-0 和 web-1 将继续使用它的主机名提供服务。

最后删除 nginx Service：

```
kubectl delete service nginx
```

```
service "nginx" deleted
```

并且删除 web StatefulSet：

```
kubectl delete statefulset web
```

```
statefulset "web" deleted
```

Pod 管理策略 {#pod-management-policy}

对于某些分布式系统来说，StatefulSet 的顺序性保证是不必要和/或者不应该的。这些系统仅仅要求唯一性和身份标志。

你可以指定 Pod 管理策略 以避免这个严格的顺序； 你可以选择 OrderedReady（默认）或 Parallel。

OrderedReady Pod 管理策略 {#orderedready-pod-management}

OrderedReady Pod 管理策略是 StatefulSet 的默认选项。它告诉 StatefulSet 控制器遵循上文展示的顺序性保证。

当你的应用程序需要或期望变更（例如推出应用程序的新版本）按照 StatefulSet 提供的序号（Pod 编号）的严格顺序发生时，请使用此选项。换句话说，如果你已经有了 Pod app-0、app-1 和 app-2，Kubernetes 将首先更新 app-0 并检查它。一旦检查良好，Kubernetes 就会更新 app-1，最后更新 app-2。

如果你再添加两个 Pod，Kubernetes 将设置 app-3 并等待其正常运行，然后再部署 app-4。

因为这是默认设置，所以你已经在学习使用它，本教程不会让你再次执行类似的步骤。

Parallel Pod 管理策略 {#parallel-pod-management}

另一种选择，Parallel Pod 管理策略告诉 StatefulSet 控制器并行的终止所有 Pod，在启动或终止另一个 Pod 前，不必等待这些 Pod 变成 Running 和 Ready 或者完全终止状态。

Parallel Pod 管理选项仅影响扩缩容操作的行为。变更操作不受其影响；Kubernetes 仍然按顺序推出变更。对于本教程，应用本身非常简单：它是一个告诉你其主机名的网络服务器（因为这是一个 StatefulSet，每个 Pod 的主机名都是不同的且可预测的）。

这份清单和你在上文下载的完全一样，只是 web StatefulSet 的 .spec.podManagementPolicy 设置成了 Parallel。

在一个终端窗口监视 StatefulSet 中的 Pod。

```
# 让 watch 一直运行直到本节结束
kubectl get pod -l app=nginx --watch
```

在另一个终端中，重新配置 StatefulSet 以进行 Parallel Pod 管理：

```
kubectl apply -f https://k8s.io/examples/application/web/web-parallel.yaml
```

```
service/nginx updated
statefulset.apps/web updated
```

保持你运行监视进程的终端为打开状态，并在另一个终端窗口中扩容 StatefulSet：

```
kubectl scale statefulset/web --replicas=5

statefulset.apps/web scaled
```

在 `kubectl get` 命令运行的终端里检查它的输出。它可能看起来像：

```
web-3 0/1 Pending 0 0s
web-3 0/1 Pending 0 0s
web-3 0/1 Pending 0 7s
web-3 0/1 ContainerCreating 0 7s
web-2 0/1 Pending 0 0s
web-4 0/1 Pending 0 0s
web-2 1/1 Running 0 8s
web-4 0/1 ContainerCreating 0 4s
web-3 1/1 Running 0 26s
web-4 1/1 Running 0 2s
```

StatefulSet 启动了三个新的 Pod，而且在启动第二和第三个之前并没有等待第一个变成 Running 和 Ready 状态。

如果你的工作负载具有有状态元素，或者需要 Pod 能够通过可预测的命名来相互识别，特别是当你有时需要快速提供更多容量时，此方法非常有用。如果本教程的这个简单 Web 服务突然每分钟收到额外 1,000,000 个请求，那么你可能会想要运行更多 Pod，但你也不想等待每个新 Pod 启动。并行启动额外的 Pod 可以缩短请求额外容量和使其可供使用之间的时间。

你应该打开两个终端，准备在清理过程中运行 `kubectl` 命令。

```
kubectl delete sts web
# sts 是 statefulset 的缩写
```

你可以监视 `kubectl get` 来查看那些 Pod 被删除：

```
# 当你看到需要的内容后结束 watch
kubectl get pod -l app=nginx --watch
```

```

web-3 1/1 Terminating 0 9m
web-2 1/1 Terminating 0 9m
web-3 1/1 Terminating 0 9m
web-2 1/1 Terminating 0 9m
web-1 1/1 Terminating 0 44m
web-0 1/1 Terminating 0 44m
web-0 0/1 Terminating 0 44m
web-3 0/1 Terminating 0 9m
web-2 0/1 Terminating 0 9m
web-1 0/1 Terminating 0 44m
web-0 0/1 Terminating 0 44m
web-2 0/1 Terminating 0 9m
web-2 0/1 Terminating 0 9m
web-2 0/1 Terminating 0 9m
web-1 0/1 Terminating 0 44m
web-1 0/1 Terminating 0 44m
web-1 0/1 Terminating 0 44m
web-0 0/1 Terminating 0 44m
web-0 0/1 Terminating 0 44m
web-0 0/1 Terminating 0 44m
web-3 0/1 Terminating 0 9m
web-3 0/1 Terminating 0 9m
web-3 0/1 Terminating 0 9m

```

在删除过程中，StatefulSet 将并发的删除所有 Pod，在删除一个 Pod 前不会等待它的顺序后继者终止。

关闭 kubectl get 命令运行的终端并删除 nginx Service：

```

kubectl delete svc nginx

```

删除本教程中用到的 PersistentVolume 卷的持久化存储介质：

```

kubectl get pvc

```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
www-web-0	Bound	pvc-2bf00408-d366-4a12-bad0-1869c65d0bee	1Gi	RWO	
standard	25m				
www-web-1	Bound	pvc-ba3bfe9c-413e-4b95-a2c0-3ea8a54dbab4	1Gi	RWO	
standard	24m				
www-web-2	Bound	pvc-cba6cfa6-3a47-486b-a138-db5930207eaf	1Gi	RWO	

```
standard    15m
www-web-3   Bound   pvc-0c04d7f0-787a-4977-8da3-d9d3a6d8d752 1Gi    RWO
standard    15m
www-web-4   Bound   pvc-b2c73489-e70b-4a4e-9ec1-9eab439aa43e 1Gi    RWO
standard    14m
```

```
kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pvc-0c04d7f0-787a-4977-8da3-d9d3a6d8d752	1Gi	RWO	Delete	Bound	
default/www-web-3	standard	15m			
pvc-2bf00408-d366-4a12-bad0-1869c65d0bee	1Gi	RWO	Delete	Bound	
default/www-web-0	standard	25m			
pvc-b2c73489-e70b-4a4e-9ec1-9eab439aa43e	1Gi	RWO	Delete	Bound	
default/www-web-4	standard	14m			
pvc-ba3bfe9c-413e-4b95-a2c0-3ea8a54dbab4	1Gi	RWO	Delete	Bound	
default/www-web-1	standard	24m			
pvc-cba6cfa6-3a47-486b-a138-db5930207eaf	1Gi	RWO	Delete	Bound	
default/www-web-2	standard	15m			

```
kubectl delete pvc www-web-0 www-web-1 www-web-2 www-web-3 www-web-4
```

```
persistentvolumeclaim "www-web-0" deleted
persistentvolumeclaim "www-web-1" deleted
persistentvolumeclaim "www-web-2" deleted
persistentvolumeclaim "www-web-3" deleted
persistentvolumeclaim "www-web-4" deleted
```

```
kubectl get pvc
```

No resources found in default namespace.

你需要删除本教程中用到的 PersistentVolume 卷的持久化存储介质。

基于你的环境、存储配置和制备方式，按照必需的步骤保证回收所有的存储。